Guardian Inputs for Shared Report Parts

Contributions to Part 0 (System Overview) and Part 4 (Test Book)

The Guardian Layer Engineering Team

Version 1.0 | November 2026

Abstract

This document compiles the Guardian engineering team's modular contributions to the collective 20-page 3-layer urban monitoring platform report. It provides the exact inputs required for **Part 0 (Shared System Overview: End-to-End Operational Scenario)** and **Part 4 (Shared Test Book: Guardian Edge Verification Scenarios G-01 through G-05)**, specifying objective verification criteria, cryptographic test vectors, and terminal reproduction commands.

1 Part 0 Input: The Guardian's Role in End-to-End Operation

1.1 End-to-End Scenario: Urban Flash Flood Detection and Gate Control (Bab Alioua Watershed)

In an active municipal stormwater event, raw water level, pump station, and rain gauge telemetry stream at 10 Hz from field transmitters into the platform. The Guardian intercepts this untrusted ingest stream at the broker perimeter, immediately validating Ed25519 digital payload signatures against public keys dynamically resolved from the Digital Twin's FastMCP service and routing unauthenticated or corrupted packets to the Dead-Letter Queue (`telemetry-dlq`). Validated telemetry is buffered into non-blocking sliding memory windows, where the Guardian computes Haar wavelet decompositions, Robust Z-scores via Median Absolute Deviation, and spatial k-NN Haversine peer differences. Within 102 ms (p_{50}), the Guardian's dual-horizon XGBoost models classify the readings, distinguishing physical water surges from sensor hardware malfunctions (such as stuck floats or electrical impulse spikes) and regional weather trends. Verified sensor readings, accompanied by confidence-scored anomaly verdicts, are published to `telemetry-validated` and ingested by the City Brain to execute automated stormwater barrier deployment and emergency evacuation alerts.

2 Part 4 (Test Book) Input: Guardian Verification Scenarios

The following five end-to-end scenarios constitute the Guardian's contribution to the platform-wide Test Book. Each scenario validates an explicit security, filtering, or analytical boundary under simulated adversarial or degraded conditions.

2.1 Scenario G-01: Cryptographic Ingestion Gate & Signature Mismatch Isolation

- **Objective:** Verify that payloads with invalid, tampered, or forged signatures are rejected at the edge before entering state memory.
- **Input Payload:** Telemetry JSON record where the Base64 signature `sig` does not match the Ed25519 signature over canonical sorted payload bytes:

```
{"device_id": "sensor-water-01", "ts": "2026-11-05T12:00:00Z", "value": 3.42, "
    unit": "m", "seq": 101, "status": "ok", "sig": "
    aW52YWxpZF9zaWduYXR1cmVfZm9yZ2Vk"}
```

- **Expected Output:** `VerifySignature` ProcessFunction rejects the payload to the side-output tag routing to `telemetry-dlq`. The emitted record contains:

```
{"error_code": "E200_BAD_SIGNATURE", "detail": "signature mismatch", "raw_payload
    ": "..."}
```

  The message is never emitted to `telemetry-validated`, and buffer state remains unmutated.

- **How to Reproduce:**

```
uv run pytest tests/consumer_producer/ -k test_bad_signature
```

2.2 Scenario G-02: Rogue / Unregistered Edge Device Key Rejection

- **Objective:** Ensure devices lacking provisioning or valid public keys in the Digital Twin key service are quarantined immediately.
- **Input Payload:** Inbound sensor payload presenting an unknown identifier:

```
{"device_id": "unregistered-node-999", "ts": "2026-11-05T12:00:00Z", "value":
    1.15, "unit": "m", "seq": 1, "status": "ok", "sig": "..."}
```

- **Expected Output:** FastMCP client calls `get_device_key("unregistered-node-999")` and receives null; `DeviceKeyProvider` raises an unknown device error. The payload is quarantined to `telemetry-dlq` with structured error code `E201_UNKNOWN_DEVICE`.
- **How to Reproduce:**

```
uv run pytest tests/consumer_producer/ -k test_unknown_device
```

2.3 Scenario G-03: Live Short-Window Anomaly Classification (Impulse Spike)

- **Objective:** Validate that high-frequency physical anomalies are correctly identified and classified within the 20-second live horizon.
- **Input:** 200 readings (20 seconds at 10 Hz) generated by `mocks/telemetry_generator.py` modeling water levels, with a synthetic single-point impulse spike injected into reading index 145 ($+4.8\sigma$ deviation).
- **Expected Output:**
  a. `StaticCheck.out_of_range` verifies the value remains within extreme electrical bounds.
  b. `PreProcess` extracts an elevated Robust Z-score ($Z = 4.82$) and elevated Haar wavelet detail energy.
  c. `AnomalyDetector` scores the feature vector in < 125 ms, outputting:

```
Verdict(sensor_id="sensor-water-01", score=0.982, label="sensor_malfunction")
```

- **How to Reproduce:**

```
python -m mocks.telemetry_generator --samples 200 --anomaly-type spike --out
    telemetry.jsonl
uv run pytest tests/ai/inference/ -k test_short_window_spike
```

2.4 Scenario G-04: Multi-Hour Drift Detection vs. Spatial Cluster Environmental Shift

- **Objective:** Confirm that the 24-hour long-window model distinguishes true hardware calibration drift from regional environmental shifts affecting all peer sensors.
- **Input:** 24-hour batch window (288 readings at 300 s intervals) across a cluster of 5 water stations. Sensors 1 through 4 exhibit a shared natural flood trend of +0.12 m/hour. Sensor 5 exhibits an anomalous sensor drift of +0.48 m/hour.
- **Expected Output:**
  a. `LongPreProcess` calculates cluster median slope median(slope) = 0.12.
  b. For Sensors 1–4, cluster slope delta is 0.00, suppressing false alarms (verdict: `Normal`).
  c. For Sensor 5, cluster slope delta is 0.36; `LongAnomalyDetector` classifies the window as `sensor_malfunction` with a drift indicator.
- **How to Reproduce:**

```
uv run pytest tests/pipelines/ -k test_long_preprocess_cluster_slope
```

2.5 Scenario G-05: Fail-Safe In-Memory Buffer Rollback on Downstream Backpressure

- **Objective:** Verify zero telemetry loss during downstream database or channel stalls via atomic rollback.
- **Input:** Continuous stream of 250 readings ingested into `BufferManager` with high_watermark = 200 and flush_size = 100. The downstream `FlushChannel` is artificially saturated (`queue.Full`).
- **Expected Output:**
  a. Capacity trigger extracts a 100-reading `FlushedBatch`.
  b. `_sink_to_queue` catches `queue.Full`.
  c. `buffer.restore(batch.readings)` atomically prepends the 100 readings back to the head of the buffer.
  d. Total buffer length recovers to 200 readings; no records dropped from process memory.
- **How to Reproduce:**

```
uv run pytest tests/managers/test_buffer_manager_v2.py -k
    test_bench_add_with_flush
```